CN PRACTICAL SLIP SOLUTIONS

by

Gaurang Dalal

SLIP 1

1. **To study the configuration of IP and MAC address for practical use.**

Q1. Turn on PC, find and note down the IPv4 Address, subnet mask, default gateway and physical MAC address. Show it to examiner

Ans:

1. Go to terminal or type **`cmd`** on search window and press enter.
2. Type **`ipconfig /all`** in the terminal window and hit enter.
3. You'll see all the details there, note them.

IPv4 Address: **`172.16.56.110`**

Subnet Mask: **`255.255.240`**

Default Gateway: **`172.16.0.1`**

Physical (MAC) Address: **`2A-A0-6B-AF-82-64`**

Q2. Write down steps to configure new IP address.

Ans:

1. Open Control Panel.
2. Go to Network and Sharing Center.
3. Click Change Adapter Settings.
4. Right click and select Properties.
5. Select Internet Protocol Version 4 (IPv4)
6. Click on Properties.
7. Select Use the following IP address.
8. Fill up the inputs accordingly.
9. Click OK.

Q3. Configure your PC with new IPv4 _____.

Ans:

- Go through the steps in Q2.
- Ip Address. Subnet Mask and Default Gateway will be provided by examiner.

Q4. Verify the configuration also check the connectivity and show it to examiner. Note the observation.

1. Go to terminal or type **`cmd`** on search window and press enter.
2. Type **`ipconfig /all`** in the terminal window and hit enter.
3. You'll see all the details there, note them.

Q5. What is difference between Logical address and physical address.

Comparison	Logical Address	Physical Address
Definition	The address used by the network layer to identify a device on network.	The unique hardware address assigned to Network Interface Card (NIC)
Also known as	IP Address (Internet Protocol)	MAC Address (Media Access Control)
OSI Layer	Network Layer	Datalink Layer
Format	32 bits (IPv4) 128 bits (IPv6)	48 bits (Hexadecimal)
Changeability	Dynamic: Changes when device connects to different network.	Fixed: It is permanent and remains same.

Q6. State the use of `ping` command.

Ans: PING stands for Packet Inter-Network Gropper. It is used check whether the destination system is reachable through the IP address and also calculates the time taken to reach the message to destination system.

To use PING command, go to terminal and type: **`ping`** *`ip_address`*

END

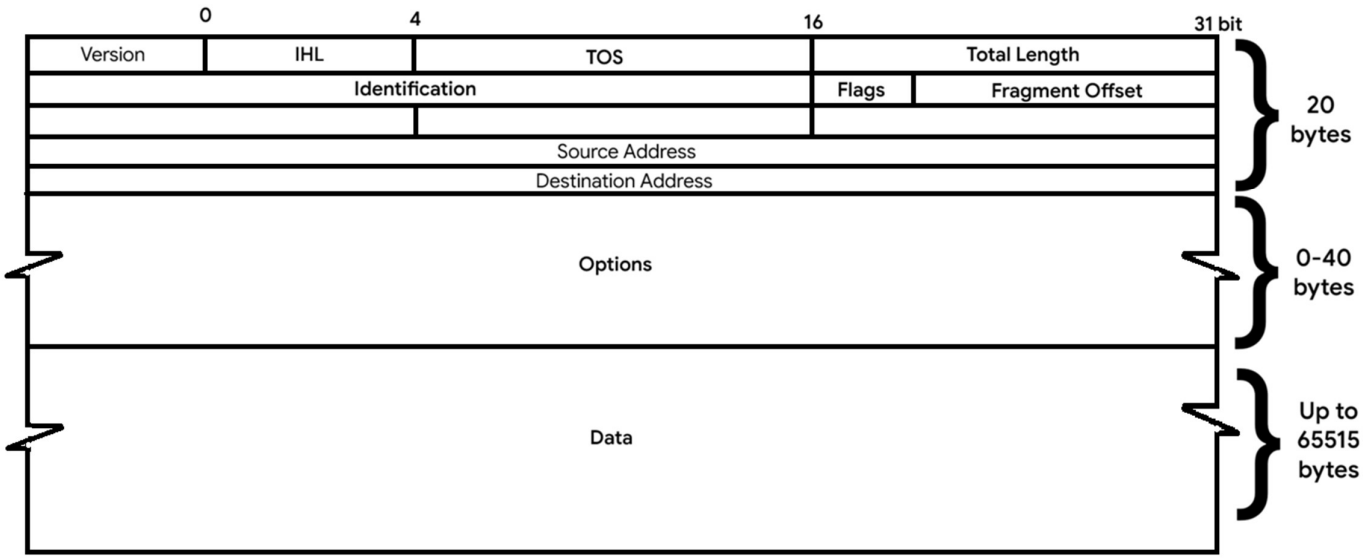
CANCELLED

SLIP 3

3. To convert the given IP addresses from dotted-decimal notion to binary notation.

Q1 Draw the IPv4 header format.

Ans:



Q2. Turn on the PC, find and note down the IPv4 Address, Subnet mask, default Gateway and Physical (MAC) address. Show it to examiner.

Ans:

1. Go to terminal or type **`cmd`** on search window and press enter.
2. Type **`ipconfig /all`** in the terminal window and hit enter.
3. You'll see all the details there, note them.

IPv4 Address: **`172.16.56.110`**

Subnet Mask: **`255.255.240`**

Default Gateway: **`172.16.0.1`**

Physical (MAC) Address: **`2A-A0-6B-AF-82-64`**

Q3. Convert the IPv4 address of your PC in to binary notation.

Ans: There are two methods to convert decimal to binary.

1. Divide the number by 2 until it is indivisible and note down the remainders as they are the new binary numbers.
2. Another method is to subtract the number with the power of the digit (128, 64, 32, 8, 4, 2, 1) until you reach zero.
3. Now figure it out yourself because I'm not putting steps-by-steps method to solve it.

Here is the answer of my IP Address: `10101100.00000001.000111000.01101110`

Q4. Convert the given (and provided by examiner) IPv4 address in to binary notation, also identify the wrong IPv4 address if any:

a. 148.162.1.12 -> `10010100.10100010.00000001.00001100`
b. 258.2.1.22 -> ***Invalid. Because the first octet is 258 and the max octet number allowed is 255.***
c. 127.256.0.1 -> ***Invalid. Because the first octet is 258 and the max octet number allowed is 255.***
d. 12.11.22.2 -> `00001100.00001011.00010110.00000010`
e. *(will provided by examiner itself).*

Q5. What is the range of value for each octet in an IPv4 address?

Ans: The range of each octet is **0 to 255.**

Minimum value: 00000000 bits.
Maximum value: 11111111 bits.

Q6. Write the total address space of IPv4 address.

Ans: The total address space of an IPv4 address is approximately **4.3 billion** addresses.
Where ***n*** is the number of bits.

- **Calculation:** 2^{32}
- **Exact Value: 4,294,967,296** addresses.

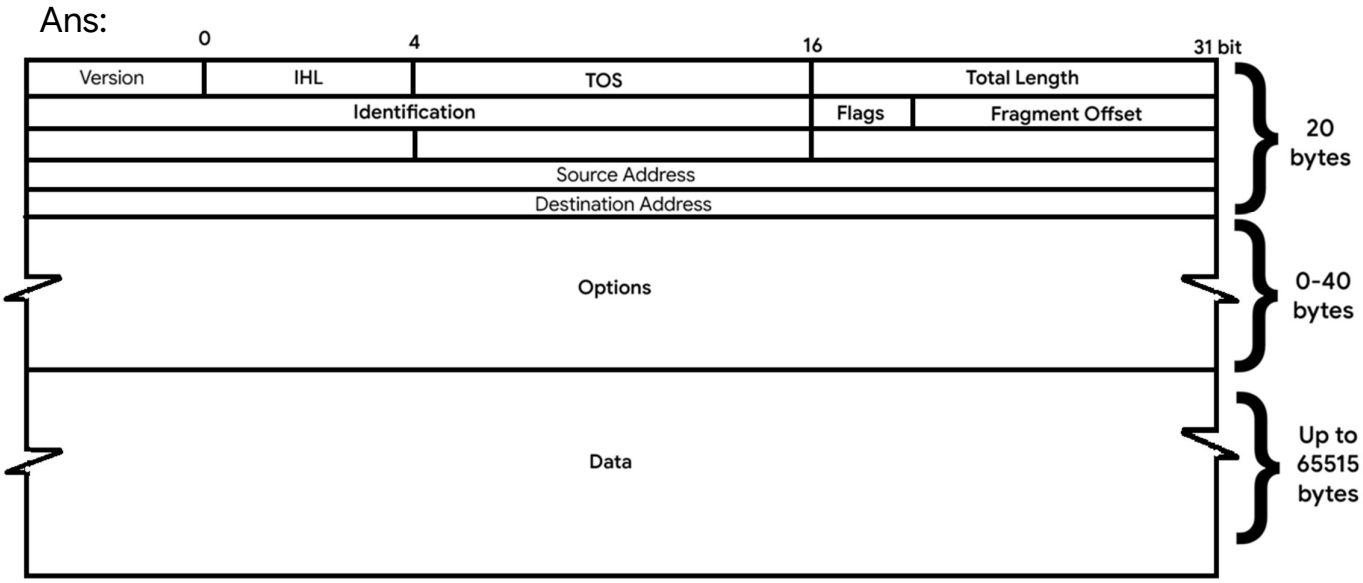
END

SLIP 4

4. To convert the given IP addresses from binary to dotted decimal notion.

Q1. Draw the IPv4 header format.

Ans:



Q2. Turn on the PC, find and note down the IPv4 Address, Subnet mask, default Gateway and Physical (MAC) address. Show it to examiner.

Ans:

1. Go to terminal or type **`cmd`** on search window and press enter.
2. Type **`ipconfig /all`** in the terminal window and hit enter.
3. You'll see all the details there, note them.

IPv4 Address: **`172.16.56.110`**

Subnet Mask: **`255.255.240`**

Default Gateway: **`172.16.0.1`**

Physical (MAC) Address: **`2A-A0-6B-AF-82-64`**

Q3. Change the given (and provided by examiner) IPv4 address in to dotted decimal notation.

a) 11000000.10101000.00000001.00001010 -> ***192.168.1.10***
b) 01111111.00000000.00000000.00000000 -> ***127.0.0.0***
c) *Will provided by examiner itself.*

Q4. Find and comment on the error if any in following IP address:

a) 111.56.045.78 -> Invalid -> Because there is '045' and leading '0' is not formal practice.
b) 11100010.23.14.55 -> Invalid -> Mixed Notation, there is binary notation in between decimal notation.
c) 221.34.7.45.8 -> Invalid -> Too many octet, there are 5 octets in here.
d) 45.75.301.14 -> Invalid -> Out of range, the third octet is greater than 255.

Q5. How many bits are there in an IPv4 address?
Ans: There are **32 bits** in IPv4 address (4 octet * 8 binary notations)

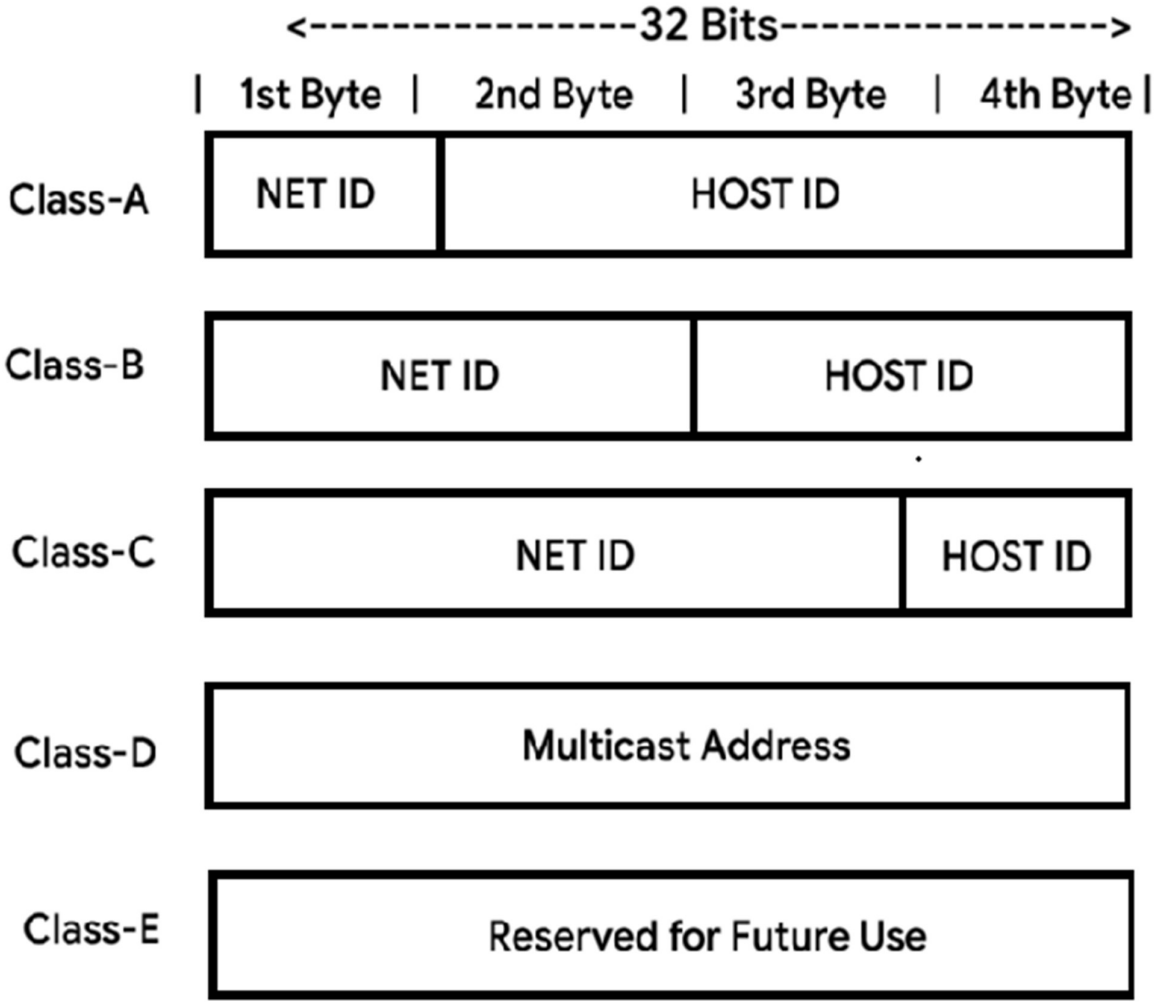
Q6. **Write the total address space of IPv4 address.**
Ans: **4,294,967,296** is the total address space of IPv4 address.

END

5. To find NetID and Host ID of the given IP addresses.

Q1. Draw the IPv4 address format in terms of Net-ID and Host-ID.

Ans:



Q2. Turn on the PC, find and note down the IPv4 Address, Subnet mask, default Gateway and Physical (MAC) address. Show it to examiner.

Ans:

1. Go to terminal or type **`cmd`** on search window and press enter.
2. Type **`ipconfig /all`** in the terminal window and hit enter.
3. You'll see all the details there, note them.

IPv4 Address: **`172.16.56.110`**

Subnet Mask: **`255.255.240`**

Default Gateway: **`172.16.0.1`**

Physical (MAC) Address: **`2A-A0-6B-AF-82-64`**

Q3. Identify the Net-ID and Host-ID of the same.

Ans:

IPv4 Address: `172.16.0.1`
Net ID: `172.16.0.0`
Host ID: `0.0.0.1`

Q4. For IP address 192.168.2.130/25, find the Net-ID and Host-ID.

Ans:

Net ID: 192.169.2.0
Host ID: 130/25

Q5. Define Net-ID and Host-ID.

Ans:

Net ID: The Net ID identifies the network to which device belongs. All devices in same network have same Net ID. It helps router to determine which network the data should sent to.

Host ID: The Host ID is device specific ID within a network where each device has its own unique Host ID. It helps identifying individual devices in that network.

Q6. How many usable hosts are there in a /28 network?

Ans:

An IPv4 address has a total of 32 bits. The /28 prefix tells that 28 bits are already reserved for the network.

- **Formula:** *total bits – network bits = host bits.*
- **Calculation:** *32 – 28 = 4 bits for* ***Host.***

1. **Calculate total hosts:**

   Total IP addresses: $2^4 = 16$

2. **Subtract reserved addresses.**

   1 address -> **Network ID**

   1 address -> **Broadcast ID**

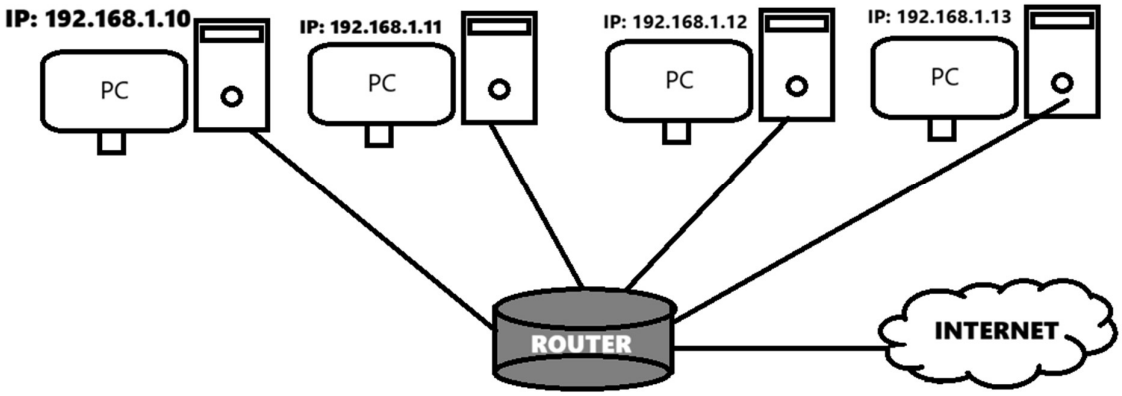
Usable host = 16 − 2 = 14

END

SLIP 6

Q1. Draw the network connecting atleast 4 PC with any classes of IP address. (Mention the IP address).

Ans:



Q2. Turn on the PC, find and note down the IPv4 Address, Subnet mask, default Gateway and Physical (MAC) address. Show it to examiner.

Ans:

1. Go to terminal or type **`cmd`** on search window and press enter.
2. Type **`ipconfig /all`** in the terminal window and hit enter.
3. You'll see all the details there, note them.

IPv4 Address: **`172.16.56.110`**

Subnet Mask: **`255.255.240`**

Default Gateway: **`172.16.0.1`**

Physical (MAC) Address: **`2A-A0-6B-AF-82-64`**

Q3. Find the class of your PC's IP address.

Ans: IP address of my PC: **172.15.56.110**

Class of my PC's IP address: **Class B**

Q4. What are the start and end address of different class of IP address?

Find the class of following IP address:

a. 10.5.25.7
b. 192.168.1.100
c. 203.5.113.25

Ans:

The start and end address of **Class A** is **1.0.0.0 and 126.255.255.255.**

The start and end address of **Class B** is **127.0.0.0 and 191.255.255.255.**

The start and end address of **Class C** is **192.0.0.0 and 223.255.255.255.**

The start and end address of **Class D** is **224.0.0.0 and 239.255.255.255.**

The start and end address of **Class E** is **240.0.0.0 and 255.255.255.255.**

Classes of following IP addresses:

1. 10.5.25.7 -> **Class A.**
2. 192.168.1.100 -> **Class C.**
3. 203.5.113.25 -> **Class C.**

Q5. Which bits determine the class of IP address?

Ans: In IPv4 addressing, the class of IP address is determined by the **leading bits** of the first octet. These leading bits tells in which class, the IP address belongs.

For e.g., If leading bits is in between 0 to 126 then the class of the IP address will be Class A.

Q6. What is default subnet mask of Class A, B and C?

Ans:

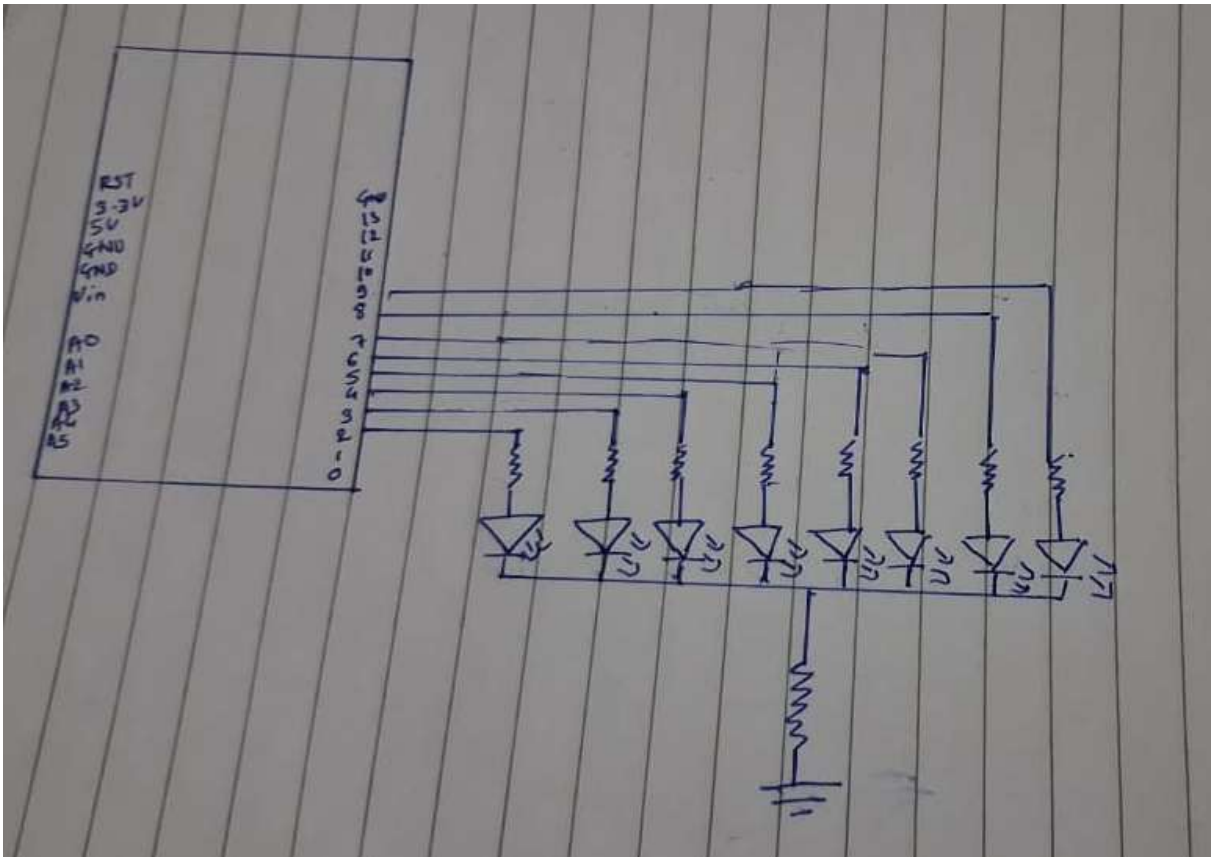
Class	Default Subnet Mask
Class A	255.0.0.0
Class B	255.255.0.0
Class C	255.255.255.0

END

SLIP 7

Q1. Draw the circuit / block diagram of LED pattern generation using Arduino and get it checked from the examiner.

Ans:



Q2. Write algorithm / flowchart for same.

Ans:

1. Start
2. Initialize LED pins as OUTPUT.
3. Define delay time for pattern speed.
4. Turn ON LEDs in desired sequence.
5. Wait for specified delay.
6. Turn OFF LEDs
7. Repeat step 4-6 to generate continuous pattern.
8. Stop. (or loop continuously).

Q3. Write program and get it checked from the examiner.

Ans:

```
void setup(){
    for(int i = 2; i ≤ 9; i++){
        pinMode(i, OUTPUT);
    }
}

void loop(){
    for(int i = 2; i ≤ 9; i++){
        digitalWrite(i, HIGH);
    }
    delay(500);

    for(int i = 2; i ≤ 9; i++){
        digitalWrite(i, LOW);
    }
    delay(500);

    for(int i = 2; i ≤ 9; i++){
        digitalWrite(i, HIGH);
        delay(200);
        digitalWrite(i, LOW);
    }

    for(int i = 9; i ≥ 2; i++){
        digitalWrite(i, HIGH);
        delay(200);
        digitalWrite(i, LOW);
    }

    for(int i = 2; i ≤ 9; i=+2)
{       digitalWrite(i, HIGH);
    }

    for(int i = 3; i ≤ 9; i=+2)
{       digitalWrite(i, LOW);
    }

    delay(500);

    for(int i = 2; i ≤ 9; i=+2)
{       digitalWrite(i, LOW);
    }

    for(int i = 3; i ≤ 9; i=+2)
{       digitalWrite(i, HIGH);
    }

    delay(500);

    for(int i = 2; i ≤ 9; i++){
        digitalWrite(i, LOW);
    }

}
```

Q4. Execute the program using Arduino IDE. Show the output to the examiner.

Ans: Upload the code to Arduino and do the require connections and done.

Q5. Modify the program for different LED pattern generation.

Ans: Change the delay in program for different LED generation.

Q6. What is the function of *pinMode()* in Arduino?

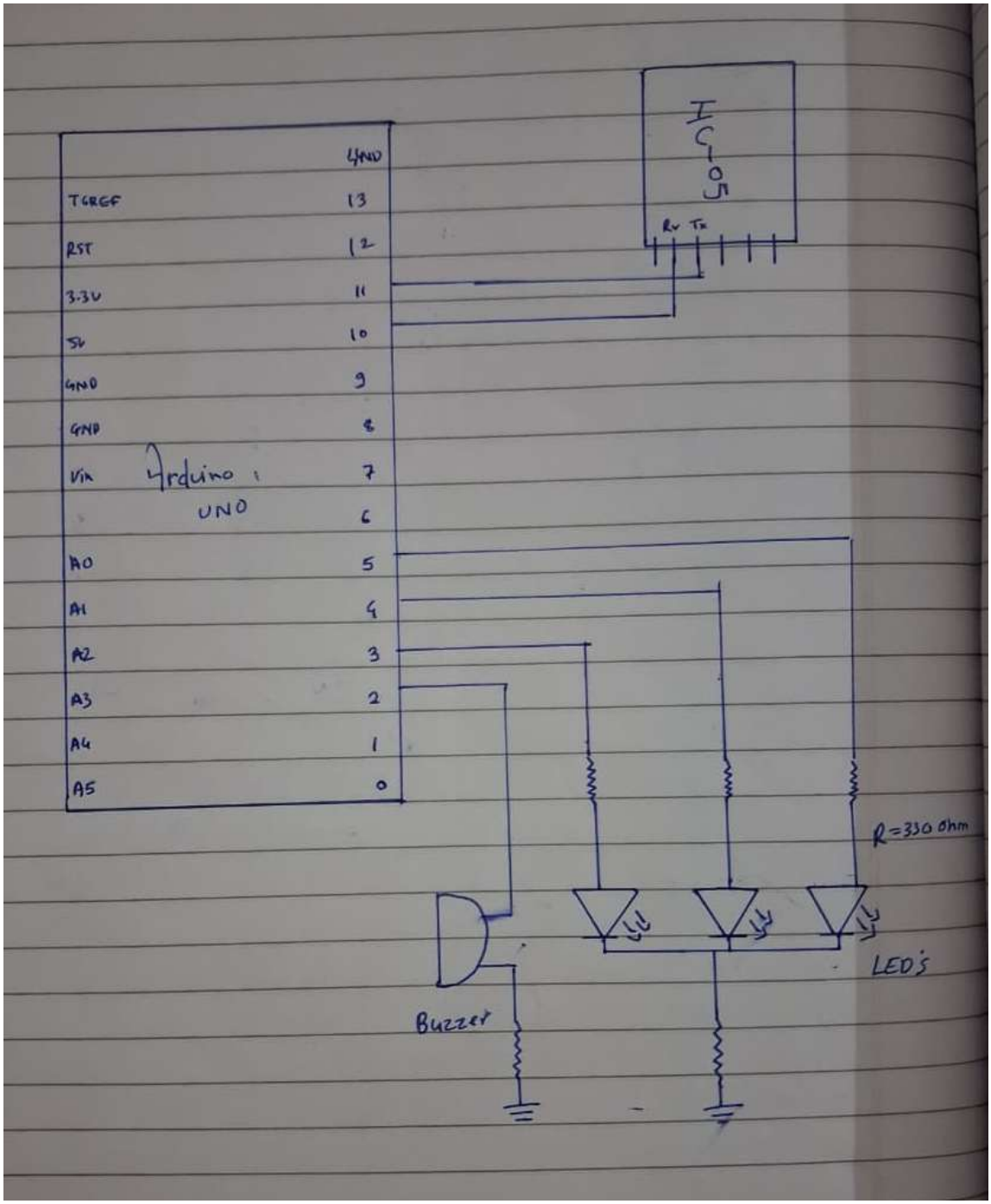
Ans: The *pinMode()* function is used to configure an Arduino pin as an input or an output.

END

SLIP 8

Q1. Draw the circuit / block diagram to interface Bluetooth module with Arduino and get it checked from examiner.

Ans:



Q2. Write Algorithm / Flowchart for same.

Ans:

1. Start.
2. Initialize Bluetooth Module.
3. Enable Bluetooth on both devices.
4. Search for nearby Bluetooth devices.
5. Select target device.
6. Pair devices using passkey.
7. Establish connection.
8. Send/Receive data.
9. Check connection status.
10. If Connection lost -> Reconnect.
11. Stop.

Q3. Write program to control LED using Bluetooth and get it checked from the examiner.

Ans:

```
#include<SoftwareSerial.h>

SoftwareSerial bluetooth(10, 11);
char data;

#define BUZZER_PIN 2
#define RED_LED 3
#define YELLOW_LED 4
#define GREEN_LED 5

void setup(){
    pinMode(RED_LED, OUTPUT);
    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(YELLOW_LED, OUTPUT);
    pinMode(GREEN_LED, OUTPUT);

    Serial.begin(9600);
    bluetooth.begin(9600);

    digitalWrite(YELLOW_LED, LOW);
    digitalWrite(RED_LED, LOW);
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(BUZZER_PIN, HIGH);

    Serial.println("Bluetooth control ready!");
}

void loop(){
    if(bluetooth.available()){
        data = bluetooth.read();

        Serial.print("Received: ");
        Serial.println(data);

        switch(data){
            case 'Y':
                digitalWrite(YELLOW_LED, HIGH);
                Serial.println("YELLOW LED ON");
                break;
```

```
            case 'y':
                digitalWrite(YELLOW_LED, LOW);
                Serial.println("YELLOW LED OFF");
                break;

            case 'B':
                buzzerBeep();
                Serial.println("Buzzer Activate
d");             break;

            case 'R':
                digitalWrite(RED_LED, HIGH);
                Serial.println("RED LED ON");
                break;

            case 'G':
                digitalWrite(GREEN_LED, HIGH);
                Serial.println("GREEN LED ON");
                break;

            default:
                Serial.println("Invalid argumen
t");             break;
        }
    }
}

void buzzerBeep(){
    for(int i = 0; i < 3; i++){
        digitalWrite(BUZZER_PIN, LOW);
        delay(200);
        digitalWrite(BUZZER_PIN, HIGH);
        delay(150);
    }
}
```

Q4. Execute the program using Arduino IDE. Show the output to the examiner.
Ans: Upload the code to Arduino and do the require connections and done.

Q5. State the application of Bluetooth.
Ans: Bluetooth can be used in fields like:

1. Smart Home and IoT (Internet of Things).
2. Wireless Audio Transmission. (Bluetooth enabled headphones)
3. Computer Peripherals. (Bluetooth Keyboard/Mouse)
4. Wearable Technology. (Smart Watches)
5. Short Range Data Transfer.

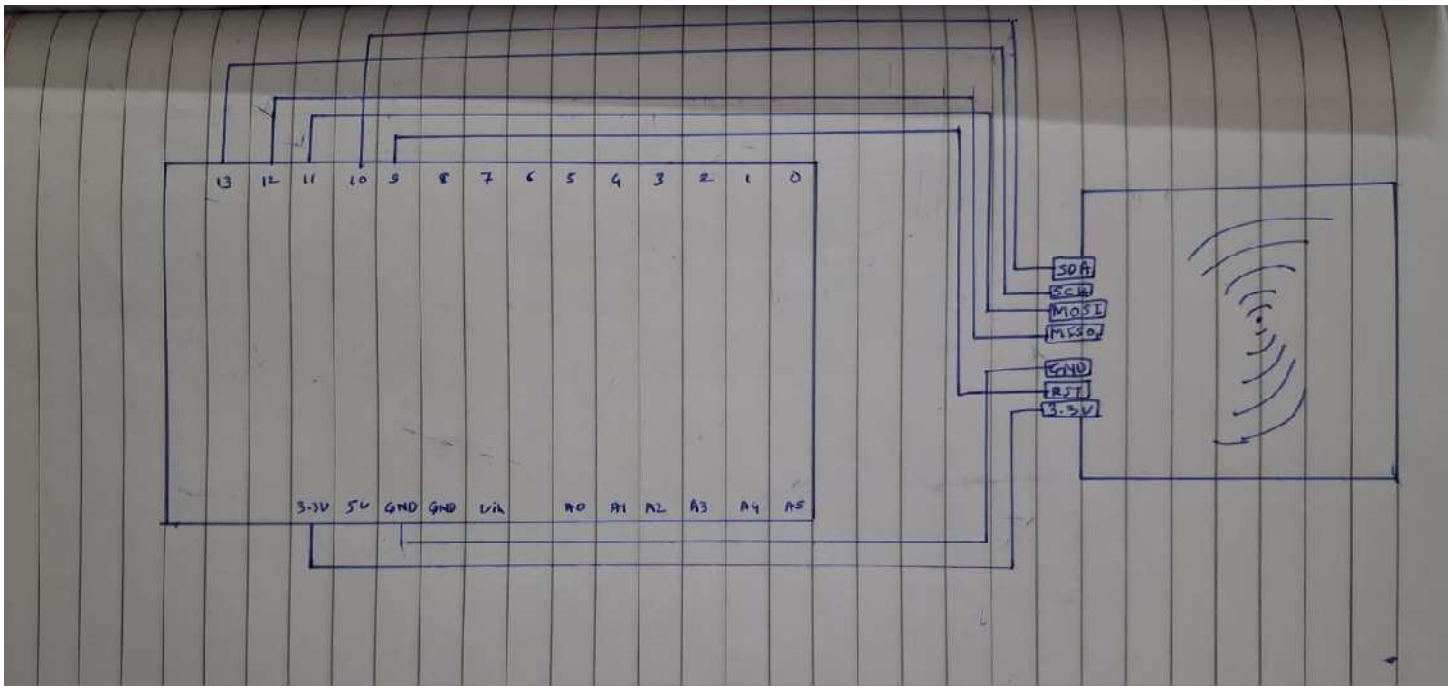
Q6. What is the range of Bluetooth device?
Ans: The range of Bluetooth device can range from 10 meters to 100 meters depending upon the device.

END

SLIP 9

Q1. Draw the circuit / block diagram to interface RFID with Arduino and get it checked from examiner.

Ans:



Q2. Write Algorithm / Flowchart for same.

Ans:

1. Start.
2. Initialize RFID reader module.
3. Wait for RFID tag near the reader.
4. Read data from RFID tag.
5. Compare tag ID with stored database.
6. IF tag is valid:
   -> Grant access / display authorized message.
7. ELSE:
   -> Deny access / show alert.
8. Repeat the process continuously.
9. Stop.

Q3 Write program to detect the RFID tag and get it checked from the examiner.

Ans:

```
#include <SPI.h>
#include <MFRC522.h>

#define SS_PIN 10
#define RST_PIN 9
#define OK_LED 3
#define ERROR_BUZ 4

MFRC522 rfid(SS_PIN, RST_PIN);
MFRC522::MIFARE_Key key;

int My_RFID_Tag[4] = {0x69, 0xD2, 0xCB, 0x06};

boolean My_Card = false;
byte nuidPICC[4];
String lastSent;

void setup(){
    pinMode(OK_LED, OUTPUT);
    pinMode(ERROR_BUZ, OUTPUT);

    digitalWrite(OK_LED, LOW);
    digitalWrite(ERROR_BUZ, HIGH);

    Serial.begin(9600);
    SPI.begin();
    rfid.PCD_Init();
}

void loop(){
    My_Card = false;

    if(!rfid.PICC_IsNewCardPresent()){
        return;
    }

    if(!rfid.PICC_ReadCardSerial()){
        return;
    }
```

```
    PrintHex(rfid.uid.uidByte, rfid.uid.size);
    Serial.println();

    My_Card = true;

    for(int i = 0; i < 4; i++){
        if(My_RFID_Tag[i] != rfid.uid.uidByte[i]){
            My_Card = false;
            break;
        }
    }

    if(My_Card){
        Serial.println("\nWelcome to class");
        Serial.println("\nAccess Granted!!");

        digitalWrite(ERROR_BUZ, HIGH);
        digitalWrite(OK_LED, HIGH);
        delay(100);
        digitalWrite(OK_LED, LOW);
    }

    else{
        Serial.println("\nGet da fck out of here!");
        Serial.println("\nAccess Denied");
        digitalWrite(OK_LED, LOW);
        digitalWrite(ERROR_BUZ, LOW);
        delay(150);
        digitalWrite(ERROR_BUZ, HIGH);

        rfid.PICC_HaltA();
        rfid.PCD_StopCrypto1();
    }
}

void PrintHex(byte *buffer, byte bufferSize){
    lastSent = "";
    for(byte i = 0; i < bufferSize; i++){
        byte n1 = (buffer[i] >> 4) & 0x0F;
        byte n2 = buffer[i] & 0x0F;
        lastSent += n1 < 10 ? '0' + n1 : 'A' + (n1 - 1
0);     lastSent += n2 < 10 ? '0' + n2 : 'A' + (n2 - 1
0);
        Serial.print(buffer[i] < 0x10 ? "0" : "");
        Serial.print(buffer[i], HEX);
    }
}
```

Q4 Execute the program using Arduino IDE. Show the output to the examiner.

Ans: Upload the code to Arduino and do the require connections and done.

Q5. State two differences between active and passive RFID tag.

Ans:

Active RFID	Passive RFID
Has its own Internal battery	No battery, powered by reader's electromagnetic field
Long range (up to 100mts)	Short Range (usually in centimetres)

Q6. State the application of RFID.

Ans:

1. Access control system (used in offices, schools, hostels, etc)
2. Attendance system.
3. Toll Collection (FASTag).
4. Asset Tracking.

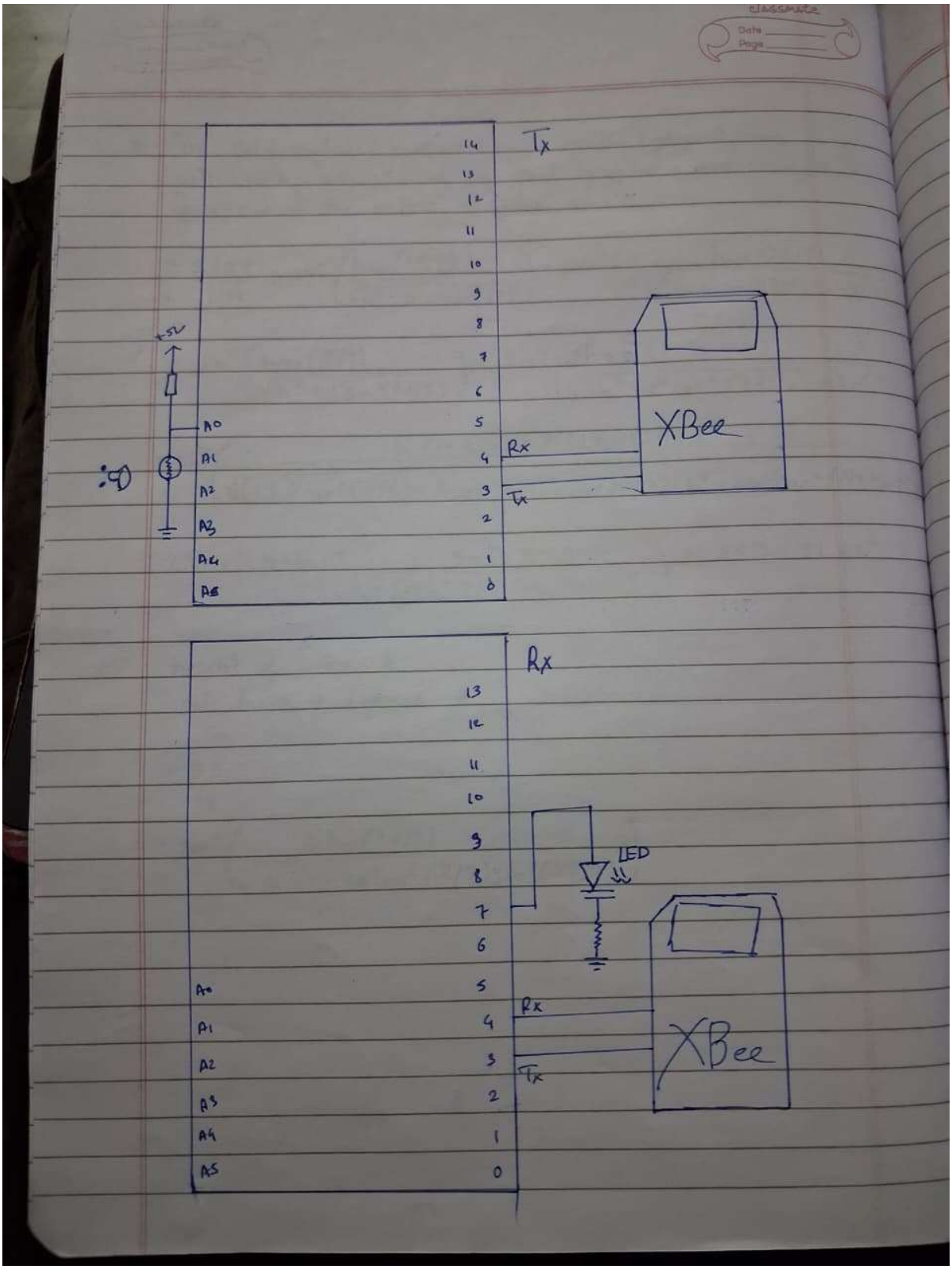
END

CANCELLED

SLIP 11

Q1. Draw the circuit / block diagram of Wireless sensor node used for street light control and get it checked from examiner.

Ans:



Q2. Write algorithm/ flowchart for same.

Ans:

1. Start.
2. Initialize microcontroller, LDR (Light Sensor), Wireless module.
3. Read light intensity from LDR.
4. IF daylight (high light intensity).
   -> Turn OFF street lights.
5. ELSE (night condition).
   -> Turn ON street lights.
6. Send status data to control station via wireless module.
7. Repeat steps continuously.

Q3. Write program for wireless sensor node used for street light control and get it checked from the examiner.

Ans:

TX

```
#include <SoftwareSerial.h>

SoftwareSerial xbee(3, 4);

int LDR = A0;
int value = 0;

void setup(){
    Serial.begin(9600);
    xbee.begin(9600);
}

void loop(){
    value = analogRead(LDR);
    xbee.println(value);

    Serial.print("Sent LDR value:
"); Serial.println(value);

    delay(1000)
}
```

RX

```
#include <SoftwareSerial.h>

SoftwareSerial xbee(3, 4);

int LED = 7
int sensorValue = 0;

void setup(){
    Serial.begin(9600);
    xbee.begin(9600);
    pinMode(LED, OUTPUT);
}

void loop(){
    if(xbee.available()){
        sensorValue = xbee.parseInt();

        Serial.print("Received LDR value: ");
        Serial.println(sensorValue);

        if(sensorValue < 500){
            digitalWrite(LED, HIGH);
            Serial.println("Street Light ON");
        }
        else{
            digitalWrite(LED, LOW);
            Serial.println("Street Light OFF");
        }
    }
}
```

Q4 Execute the program using Arduino IDE. Show the output to the examiner.
Ans: Upload the code to Arduino and do the require connections and done.

Q5. State the application of this experiment.
Ans:

1. Energy conversation.
2. Remote monitoring and control.
3. Adaptive lighting.
4. Maintained efficiency.

Q6. State the function of `Serial.begin(9600)`.
Ans: The **`Serial.begin(9600)`** function initializes serial communication between an Arduino and another device (computer) at the baud rate of ***9600.*** The 9600 baud rate act as a common and default baud rate between Tx and Rx devices for seamless data transfer.

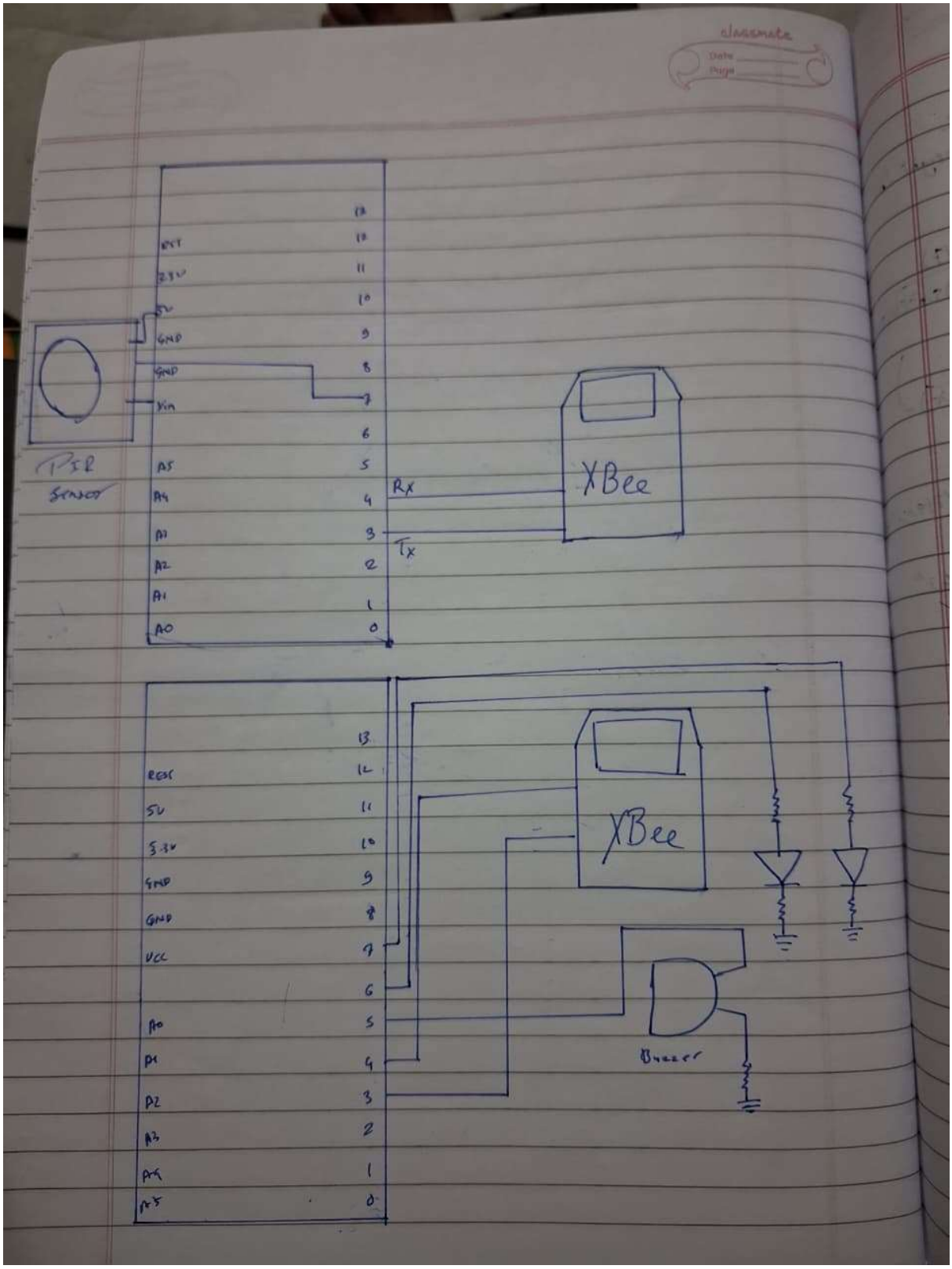
END

CANCELLED

SLIP 13

Q1 Draw the circuit / block diagram of Wireless sensor node used for security system and get it checked from examiner.

Ans:



Q2. Write algorithm/ flowchart for same

Ans:

1. Start.
2. Initialize the sensors and wireless module.
3. IF motion detected
   -> Activate alarm.
   -> Send alert to base station.
4. ELSE
   -> Continue monitoring,
5. Enter sleep mode when idle (power saving).
6. Repeat steps continuously.

Q3. Write program for wireless sensor node used for security system and get it checked from the examiner.

Ans:

```
#include <SoftwareSerial>

SoftwareSerial xbee(3, 4);

int pirPin = 7;
int lastState = LOW;

void setup(){
    pinMode(pirPin, INPUT);
    Serial.begin(9600);
    xbee.begin(9600);

    Serial.println("PIR Transimitter started");
    delay(2000);
}

void loop(){
    int sensorValue = digitalWrite(pirPin);
    if(sensorValue ≠ lastState){
        if(sensorValue == HIGH){
            xbee.println("INTRUDER_ALERT");
            Serial.println("Motion Detected! Alert Sent
🚨");   }
        else{
            xbee.println("AREA_CLEAR");
            Serial.println("Area Clear");
        }
        lastState = sensorValue;
    }
    delay(200);
}
```

```
#include <SoftwareSerial.h>

SoftwareSerial xbee(3, 4);

int redLED = 7;
int greenLED = 6;
int buzzer = 5;

String receivedMessage = "";

void setup(){
    Serial.begin(9600);
    xbee.begin(9600);

    pinMode(redLED, OUTPUT);
    pinMode(greenLED, OUTPUT);
    pinMode(buzzer, OUTPUT);

    digitalWrite(redLED, LOW);
    digitalWrite(greenLED, HIGH);
    digitalWrite(buzzer, HIGH);

    Serial.println("Receiver ready");
}

void loop(){
    if(xbee.available()){
        receivedMessage = xbee.readStringUntil
('\n'); receivedMessage.trim();

        Serial.print("Received: ");
        Serial.println(receivedMessage);

        if(receivedMessage == "INTRUDER_ALERT"){
            activateAlarm();
        }
        else if(receivedMessage == "AREA_CLEAR"){
            deactivateAlarm();
        }
    }
}

void activateAlarm(){
    digitalWrite(redLED, HIGH);
    digitalWrite(greenLED, LOW);
    digitalWrite(buzzer, LOW);
}

void deactivateAlarm(){
    digitalWrite(redLED, LOW);
    digitalWrite(greenLED, HIGH);
    digitalWrite(buzzer, HIGH);
}
```

Q4 Execute the program using Arduino IDE. Show the output to the examiner.
Ans: Upload the code to Arduino and do the require connections and done.

Q5. State the function of "`digitalWrite(9, HIGH)`"
Ans:

The `digitalWrite(9, HIGH);` is used in Arduino programming to set a specific digital pin to a high voltage state.

The digitalWrite`()` is used to control the output state of a digital pin
The 9 identifies the specific digital pin on Arduino UNO.
The `HIGH` is the value assigning to the pin.

Q6. How does the PIR sensor detect the motion?
Ans: A PIR sensor detects motion by measuring changes in **infrared radiation** as any object / human being moves across its two internal sensing zones, the PIR sensor detects the motion.

END

CANCELLED

CANCELLED

End